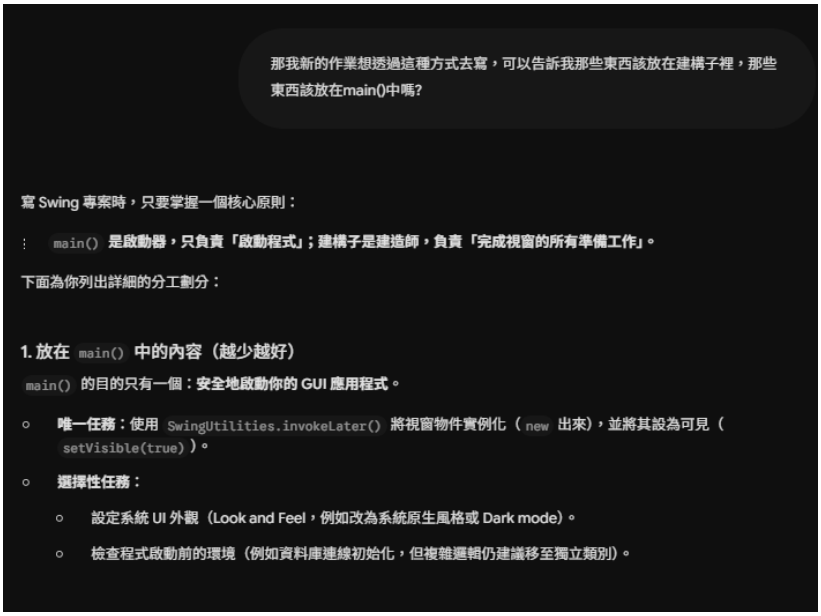
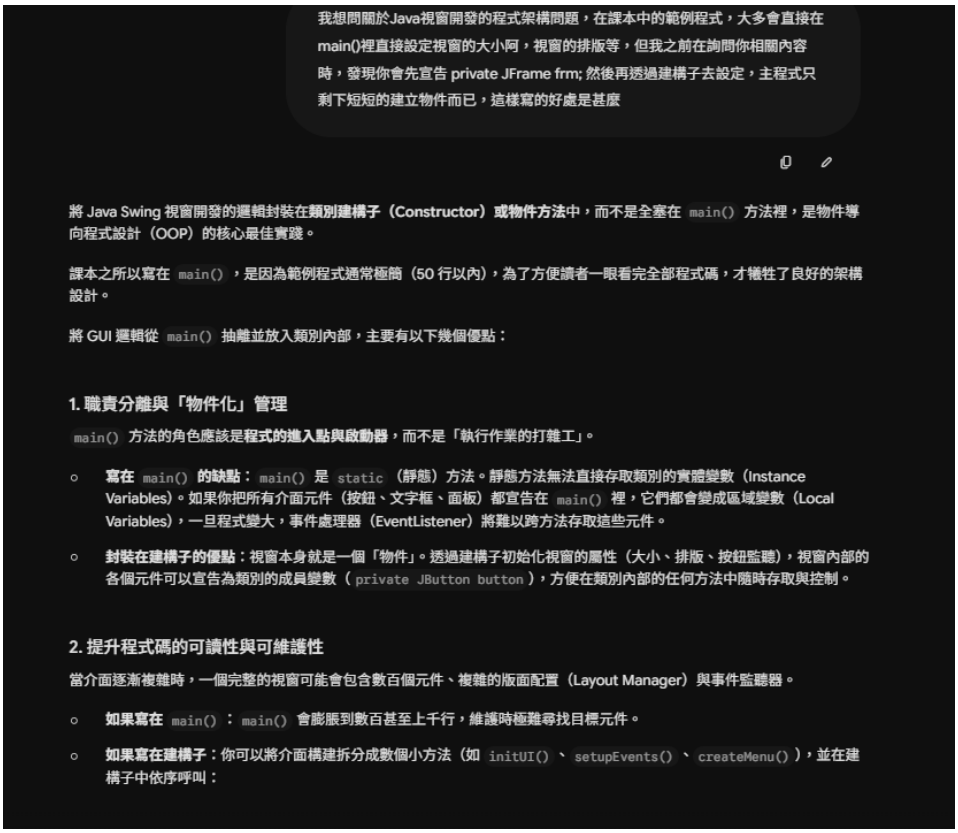
學號:A1143345 姓名:蔡承佑 作業:exercise1 AI 反思

(一)架構問題:

因為之前在透過 AI 學習這單元的內容時，發現 AI 在寫時並不會把東西都寫在 main()中，因此向 AI 詢問了相關內容





main() 範例：

Java

```
public static void main(String[] args) {
    // 1. (選擇性) 設定 UI 風格為系統原生風格
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception e) {
        e.printStackTrace();
    }

    // 2. (核心) 在 Swing 的事件派發執行緒 (EDT) 中啟動視窗
    SwingUtilities.invokeLater(() -> {
        MyMainFrame frame = new MyMainFrame(); // 建立視窗物件 (會執行建構子)
        frame.setVisible(true);              // 顯示視窗
    });
}
```

2. 放在類別成員變數（Field）中的內容

需要跨方法存取、或需要在事件監聽器（Listener）中讀取的元件，必須宣告為類別的 `private` 成員變數：

- 輸入元件（如 `JTextField`、`JTextArea`、`JComboBox`、`JCheckBox`）：因為按按鈕時需要讀取裡面的文字或狀態。
- 狀態顯示元件（如 `JLabel`、`JProgressBar`）：因為觸發事件時需要更新上面的內容。
- 核心邏輯/資料物件：例如自訂的資料庫連線物件、資料清單模型（`DefaultTableModel`）等。

註：不需在事件中存取的靜態元件（如不會變動的標題 *JLabel*、排版用的 *JPanel*），可以直接宣告在建構子或初始化方法內作為區域變數。

3. 放在建構子（Constructor）中的內容

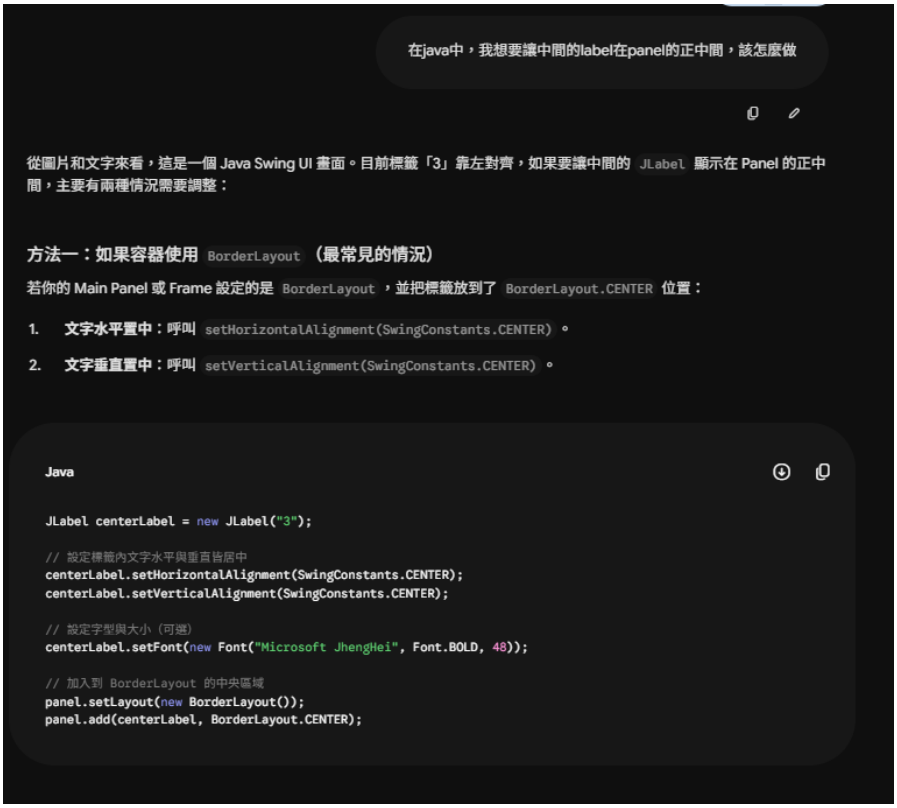
視窗第一次顯示前必須設定好的一切，都應該在建構子（或由建構子呼叫的私有方法）中完成。

為了避免建構子太長，通常會在建構子內依序呼叫 3 個私有輔助方法：

1. 視窗基本屬性設定（`initFrame()`）
   - 設定標題（`setTitle`）
   - 設定視窗大小與位置（`setSize`，`setLocationRelativeTo(null)` 置中）
   - 設定關閉行為（`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`）
   - 設定版面管理器（`setLayout`）
2. 元件初始化與排版（`initComponents()`）
   - 實例化各個 GUI 元件（`new JButton()`，`new JTextField()` 等）。
   - 將元件加入面板（`JPanel`）或視窗（`add()`）。
   - 設定元件大小、字型、顏色或邊框。
3. 事件監聽器註冊（`initEvents()`）
   - 為按鈕加入點擊事件（`button.addActionListener(e -> { ... })`）。
   - 為鍵盤、滑鼠或視窗事件綁定 Listener。

(一) 排版問題

因為在執行後發現了就算把 label 設為 CENTER，文字不會在中間而是靠左，因此詢問了 ai，學到了 setHorizontalAlignment(SwingConstants.CENTER) 的用法



(二) 改變 Label 字顏色

翻找課本後似乎沒有找到相關的方法介紹，因此詢問了 AI



(三) 其他在寫作業時的小問題

1. 傾聽者設定問題:



```java
public A1143345_excerise1(){
    frm=new JFrame("骰子模擬器");
    frm.setSize(400,300);
    frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frm.setLayout(new BorderLayout());
    pnl=new JPanel();
    pnl.setLayout(new BorderLayout());
    lab_center=new JLabel();
    lab_north=new JLabel();
    lab_center.setFont(new Font("微軟正黑體",Font.BOLD,60));
    lab_center.setHorizontalAlignment(SwingConstants.CENTER);
    lab_north.setFont(new Font("微軟正黑體",Font.BOLD,20));
    btn=new JButton("投骰子");
    btn.addActionListener(this);
    //用this，代表現在這個建構子建立的物件
    //如果在這重新new一個新物件會變無限迴圈
    frm.add(pnl);
    pnl.add(lab_center,"Center");
    pnl.add(lab_north,"North");
```

2. 程式碼擺放位置問題:



3. 忘記 random()怎麼用:





2. java.util.Random 類別

如果你需要生成多種資料型態（如 int、boolean、long）或高頻率生成隨機數，使用 Random 類別更加靈活且方便。

- 需要引入：import java.util.Random;
- 使用步驟：先建立 Random 物件，再調用對應的方法。

常用方法：

- nextInt(int bound)：生成 $[0, bound)$ 之間的整數（包含 0，不包含 bound）。
- nextDouble()：生成 $[0.0, 1.0)$ 之間的浮點數。
- nextBoolean()：隨機回傳 true 或 false。

範例程式碼：

Java

```
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        Random rand = new Random();

        // 1. 生成 0 ~ 9 的整數
        int n1 = rand.nextInt(10);

        // 2. 生成 1 ~ 100 的整數
        int n2 = rand.nextInt(100) + 1;

        // 3. 隨機布林值（例如：模擬拋硬幣）
        boolean coin = rand.nextBoolean();
    }
}
```

(四) AI 反思:

在作業中，使用 AI 的方式大多都是將其當作快速的搜尋引擎，比如說，忘記了 random()的使用方式、想知道怎麼讓 JLabel 置中，我會讓 AI 一個使用範例，然後再回到作業嘗試，這樣更有助於在借助 AI 力量的同時，保有自己的思考跟學習能力，我認為我目前的階段跟能力，還不足以「由 AI 生成，自己改錯誤」的這個模式

以下為在這次作業中，運用 AI 學到的用法或觀念:

- label.setForeground(Color.RED); 設定 label 顏色
- setHorizontalAlignment(SwingConstants.CENTER) 讓文字水平置中
- setVerticalAlignment(SwingConstants.CENTER) 讓文字垂直置中
- 嘗試讓程式職責分離，讓 main()只負責啟動，把其他東西封裝在建構子跟函數中
- String.format()的用法，可以用來把不同型態的變數插入到字串中，讓其產生一個新字串
- 打算在目前類別的建構子中設定傾聽者時，應該使用 this，而不是再創建一個類別的物件，因為這樣會造成無限重複，最後崩潰